Algorithmique Fondamentale : Recherche Séquentielle et Recherche Dichotomique dans les Tableaux

Thème principal : Méthodes de recherche dans une structure linéaire (tableau), préconditions de tri, complexité algorithmique et arbitrage opérationnel

Synthèse exécutive : Ce cours traite des deux stratégies fondamentales pour localiser un élément (une valeur cible) au sein d'un tableau en programmation : la recherche séquentielle (linéaire) et la recherche dichotomique (basée sur le paradigme 'diviser pour régner'). L'enjeu pédagogique majeur est d'établir la relation directe entre la structure préalable des données (tableau trié ou non trié) et la complexité temporelle du traitement. La recherche séquentielle explore le tableau case par case en O(n) sans exiger de précondition, tandis que la recherche dichotomique divise l'espace de recherche par deux à chaque étape en O(log n), à la condition impérative que le tableau soit préalablement trié. L'étudiant doit intégrer le fonctionnement pas à pas de ces algorithmes, la manipulation exacte des pointeurs d'indices (début, fin, milieu), ainsi que les critères d'arbitrage économique entre le coût du tri et la fréquence des requêtes de recherche.

1. La Recherche Séquentielle (ou Linéaire)

La recherche séquentielle est l'approche algorithmique la plus intuitive et universelle pour retrouver une valeur dans un tableau. L'algorithme initialise un pointeur d'indice à 0 et parcourt chaque case une à une. À chaque itération, une comparaison logique est effectuée entre la valeur contenue dans la case courante et la valeur recherchée (la cible). Deux issues sont possibles : 1. Succès : La valeur est trouvée. L'algorithme s'arrête immédiatement et renvoie l'indice correspondant. 2. Échec : La fin du tableau (indice n-1) est atteinte sans trouver la valeur. L'algorithme renvoie une valeur conventionnelle signifiant l'absence (généralement -1 ou Faux). Dans le pire des cas (si l'élément est à l'indice n-1 ou n'existe pas dans le tableau), l'algorithme effectue exactement n comparaisons pour un tableau de n éléments. Sa complexité temporelle est donc notée O(n). Son principal avantage stratégique est qu me elle ne requiert aucune organisation préalable des données.

- Parcours exhaustif et séquentiel du tableau du premier indice (0) au dernier indice (n-1)
- Absence totale de précondition : utilisable indifféremment sur tableau trié ou non trié
- Complexité temporelle linéaire O(n) au pire des cas (élément situé en toute fin ou absent)
- Arrêt précoce (anticipé) dès que l'élément cible est localisé dans la structure

Cas concret enseignant : *Analogie du professeur : La recherche d'une clé tombée dans un tas de feuilles mortes. N'ayant aucun ordre ni rangement possible dans le tas de feuilles, il n'existe aucune astuce : la seule façon garantie de trouver la clé est de soulever chaque feuille une à une, de la première jusqu'à la dernière.*

2. La Recherche Dichotomique (Principe du Diviser pour Régner)

La recherche dichotomique exploite la propriété de tri préexistante d'un tableau pour éliminer massivement des sous-ensembles d'éléments sans avoir à les examiner. L'algorithme délimite la fenêtre de recherche active grâce à deux pointeurs d'indices : 'debut' (initialisé à 0) et 'fin' (initialisé à n-1). À chaque itération de la boucle de recherche, l'algorithme calcule l'indice central par division entière : milieu = (debut + fin) / 2. Il compare ensuite la valeur T[milieu] avec la cible recherchée : 1. Si T[milieu] == cible : L'élément est trouvé, la recherche s'arrête (succès). 2. Si cible < T[milieu] : L'élément ne peut se trouver que dans la moitié gauche du sous-tableau. On restreint la zone en déplaçant le pointeur de fin : fin = milieu - 1. 3. Si cible > T[milieu] : L'élément ne peut se trouver que dans la moitié droite. On déplace le pointeur de début : debut = milieu + 1. Le processus s'itère tant que debut <= fin et que l'élément n'a pas été trouvé. En divisant l'espace de recherche par 2 à chaque étape, l'algorithme atteint une complexité logarithmique O(log2 n). Pour un tableau de 1 000 000 d'éléments triés, 20 comparaisons suffisent au maximum (car 2^20 ≈ 1 048 576), contre 1 000 000 pour la recherche séquentielle.

- Précondition stricte et obligatoire : Le tableau doit obligatoirement être strictement trié au préalable

- Paradigme algorithmique 'Diviser pour Régner' réduisant l'espace de recherche de 50 % à chaque étape
- Gestion dynamique de 3 pointeurs d'indices : debut, fin, et milieu = (debut + fin) / 2
- Complexité temporelle logarithmique O(log2 n) au pire des cas (ex: 20 comparaisons max pour 1 000 000 d'éléments)

Cas concret enseignant : *Exemple du professeur : Le jeu du 'Devine un nombre entre 1 et 100'. Proposer 1, 2, 3... est une approche séquentielle inefficace. Proposer 50 élimine d'un coup 50 possibilités selon que la réponse est 'c'est plus grand' ou 'c'est plus petit'. En poursuivant cette dichotomie, la réponse est garantie en 7 étapes maximum (car 2^7 = 128 > 100).*

3. Arbitrage Algorithmique et Analyse Comparative de Performance

En informatique, le choix d'un algorithme ne repose pas uniquement sur sa vitesse dans le cas idéal, mais sur le contexte global de son application. Bien que la recherche dichotomique présente une complexité logarithmique extrêmement favorable O(log n), elle impose la précondition d'un tableau trié. Or, trier un tableau désordonné coûte au mieux O(n log n) opérations (avec des algorithmes comme le Tri Fusion ou le Tri Rapide). Trois cas de figure guident le choix de l'algorithme : 1. Recherche ponctuelle (unique) sur tableau non trié : Exécuter directement une recherche séquentielle O(n) est plus économique que de trier d'abord puis de faire une recherche dichotomique (O(n log n + log n) > O(n)). 2. Tableau déjà trié : La recherche dichotomique en O(log n) s'impose sans hésitation. 3. Tableau statique soumis à de très nombreuses requêtes (k recherches) : Effectuer le tri initial en O(n log n) une fois pour toutes est hautement rentable, car le coût total O(n log n + k * log n) devient négligeable devant k * O(n) quand k est grand.

- Coût algorithmique du tri préalable : Trier un tableau non ordonné nécessite au mieux une complexité O(n log n)
- Règle de la recherche unique : Sur un tableau non trié, la recherche séquentielle O(n) est plus rapide que de trier puis chercher (O(n log n + log n))
- Règle des recherches multiples : L'investissement initial d'un tri O(n log n) est largement amorti par de nombreuses recherches dichotomiques ultérieures en O(log n)

Cas concret enseignant : *Analogie du professeur : L'analogie du dictionnaire imprimé. Ordonner l'intégralité des mots du dictionnaire par ordre alphabétique demande un travail considérable à l'imprimeur (le tri initial), mais cet effort unique permet ensuite à des millions d'utilisateurs de retrouver n'importe quel mot en quelques secondes pendant des années.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne pas toujours utiliser la recherche dichotomique si elle est beaucoup plus rapide que la recherche séquentielle ?
Réponse du professeur : Parce que la recherche dichotomique exige impérativement que le tableau soit trié au préalable. Si le tableau n'est pas trié et que l'on ne doit effectuer qu'une seule recherche, le coût pour trier le tableau (au minimum O(n log n)) est bien supérieur au coût d'un simple parcours séquentiel O(n). La dichotomie n'est avantageuse que si le tableau est déjà trié ou s'il subit un grand nombre de recherches.

Question étudiant : Comment calcule-t-on l'indice du milieu si le sous-tableau contient un nombre pair d'éléments ?
Réponse du professeur : En algorithmique et en programmation C++, on utilise la division entière (troncature vers le bas). L'expression `milieu = (debut + fin) / 2` avec des entiers produit automatiquement l'arrondi inférieur. Par exemple, avec debut = 0 et fin = 9 (10 éléments), milieu = (0 + 9) / 2 = 4. L'élément central retenu est celui à l'indice 4, ce qui découpe la zone de recherche de manière totalement valide pour l'algorithme.

■ Exercice & Démonstration d'Application

Énoncé : Soit le tableau trié d'entiers T = [3, 8, 12, 15, 19, 24, 31, 42, 50, 61] délimité par n = 10 éléments (indices 0 à 9). Appliquer pas à pas l'algorithme de recherche dichotomique pour localiser la valeur 42, puis écrire la fonction C++ correspondante en respectant rigoureusement les normes de codage de l'IUT d'Anglet.

Démarche de résolution :
Résolution pas à pas du déroulement de la recherche dichotomique pour valRecherchee = 42 dans T = [3, 8, 12,

15, 19, 24, 31, 42, 50, 61] : • Initialisation : - debut = 0 - fin = 9 (indice du dernier élément) - pos = -1 • Itération 1 : - Calcul du milieu : milieu = (0 + 9) / 2 = 4 (division entière). - Valeur examinée : T[4] = 19. - Comparaison : Cible (42) > T[4] (19). - Action : La valeur recherchée est strictement supérieure. Elle se trouve donc dans la moitié droite. - Mise à jour du pointeur : debut = milieu + 1 = 4 + 1 = 5. • Itération 2 : - Fenêtre active : debut = 5, fin = 9. - Calcul du milieu : milieu = (5 + 9) / 2 = 7. - Valeur examinée : T[7] = 42. - Comparaison : Cible (42) == T[7] (42). - Action : Correspondance exacte trouvée ! pos = 7. • Arrêt du traitement : La condition de boucle (pos == -1) devint fausse. La boucle se termine.

```
/*
 * Programme : Recherche Dichotomique dans un Tableau Trié
 * Auteur : Matéi de Saint Leger
 * Matière : R1.01 - Initiation à l'Algorithmique et à la Programmation
 * Date : 12/11/2026
 */

#include <iostream>
#include <vector>
using namespace std;

// Fonction de recherche dichotomique dans un tableau d'entiers trié
int rechercherDichotomique(const vector<int>& t, int valRecherchee)
{
    // CONSTANTES

    // VARIABLES
    int debut;  // Indice de début de la fenêtre active
    int fin;    // Indice de fin de la fenêtre active
    int milieu; // Indice de la position centrale
    int pos;    // Indice du résultat (-1 si absent)

    // TRAITEMENTS
    debut = 0;
    fin = static_cast<int>(t.size()) - 1;
    pos = -1;

    while ((debut <= fin) && (pos == -1))
    {
        milieu = (debut + fin) / 2;

        if (t[milieu] == valRecherchee)
        {
            pos = milieu; // Succès : élément trouvé
        }
        else if (valRecherchee < t[milieu])
        {
            fin = milieu - 1; // Restriction à la sous-partie gauche
        }
        else
        {
            debut = milieu + 1; // Restriction à la sous-partie droite
        }
    }

    return pos;
}
```

Résultat : La valeur 42 a été localisée à l'indice 7 du tableau en seulement 2 comparaisons grâce à la recherche dichotomique. En comparaison, une recherche séquentielle aurait nécessité 8 comparaisons successives (de T[0]=3 jusqu me à T[7]=42).

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la précondition indispensable et obligatoire pour pouvoir appliquer une recherche dichotomique sur un tableau ?

R : Le tableau doit obligatoirement être préalablement trié (dans l'ordre croissant ou décroissant).

Q : Quelle est la complexité temporelle dans le pire des cas d'une recherche séquentielle sur un tableau de n éléments ?

R : La complexité temporelle est linéaire, notée O(n).

Q : Combien de comparaisons au maximum la recherche dichotomique nécessite-t-elle pour localiser un élément dans un tableau trié de 1024 éléments ?

> *R : Elle nécessite au maximum 11 comparaisons (car 2^10 = 1024, soit ceil(log2(1024)) + 1 = 11 dans le pire des cas d'arrêt).*